VisionGuard

A Low-Cost Smart Cane for Visually Impaired People with Ultrasonic Obstacle Detection and Wet-Surface Warning

Project Thesis: Complete Technical Explanation

BACSE324 Embedded Systems Architecture
B.Tech Computer Science and Engineering
Vellore Institute of Technology, Vellore
Fall Semester 2026–27 · Slot E2+TE2

S Darshan (25BCE2883) · **Tanay Agarwal** (25BCE2475)
Faculty: Dr. Shabeeb Ahmed

Abstract

A white cane, the most widely used mobility aid for visually impaired people, detects obstacles only on contact, gives no distance information and cannot warn about water, puddles or open drains. This thesis presents **VisionGuard**, a low-cost electronic cane built around an Arduino Nano (ATmega328P). An HC-SR04 ultrasonic sensor measures the distance to obstacles ahead using echo timing, and a resistive moisture probe at the tip, read through an LM393 module and the microcontroller's 10-bit ADC, detects water or wet ground. The firmware uses a non-blocking state machine with three states: WATER, which gives a continuous tone and always has priority; OBSTACLE, which gives beeps whose rate rises as the obstacle comes within 50 cm; and CLEAR. Reliability features include a median filter on distance, averaging with hysteresis on moisture, a start-up self-test, a heartbeat chirp, and a low-battery warning that measures the supply voltage using the chip's internal 1.1 V reference without extra hardware. The prototype runs from a single 18650 Li-ion cell, and the hardware on the cane costs about ₹876. Bench tests confirm obstacle detection on battery power; water-threshold calibration and testing of the new firmware are in progress. The thesis explains every component, connection, line of code and underlying principle, from basic concepts to the microcontroller's architecture.

Contents

1 Introduction

2 Background and Related Work

3 System Requirements and Design

4 Hardware Components

5 Circuit, Wiring and Power

6 The ATmega328P Microcontroller

7 Software Design

8 Physics and Mathematics

9 Build Methodology

10 Testing and Calibration

11 Results, Discussion and Limitations

12 Future Work

13 Conclusion

References

A Appendix A: Complete Source Code

B Appendix B: Bill of Materials

C Appendix C: A–Z Glossary

D Appendix D: Quick Viva Answers

1. Introduction

1.1 Background

The World Health Organization estimates that at least 2.2 billion people live with a near or distance vision impairment [6]. For independent movement, most visually impaired people rely on the white cane. It is cheap and simple, but it has three basic limitations: it detects an obstacle only when it touches it (about one cane length ahead), it gives no information about how far an obstacle is, and it cannot warn about hazards such as water, puddles, wet mud or open drains until the user steps into them.

1.2 Motivation

Electronic travel aids exist, but they are often expensive, imported and difficult to repair locally. An embedded system built from common, low-cost parts can add early warning and wet-surface detection to an ordinary cane. This also suits an Embedded Systems Architecture course, because it combines digital timing, analog measurement, real-time decision logic and power design on a single microcontroller.

1.3 Problem Statement

To design and build a low-cost, battery-powered cane that (i) detects obstacles before contact and indicates their distance, and (ii) warns the user when the tip touches water or wet ground, with the water warning taking priority.

1.4 Objectives

1. Detect obstacles within **50 cm** using an HC-SR04 ultrasonic sensor.
2. Encode distance in the beep rate: the gap between beeps varies linearly from **50 ms** (very close) to **400 ms** (at 50 cm).
3. Detect water at the tip when the moisture reading falls below a calibrated threshold (default **500 of 1023**), with a continuous tone that has **priority**.
4. Run from one **18650 Li-ion cell** with an on/off switch, with cane hardware costing under **₹900**.
5. Add reliability and usability features: a self-test, a heartbeat, a low-battery warning and filtered readings.

1.5 Scope

The prototype covers obstacle detection ahead of the cane and contact-based water detection, with audio feedback. Vibration feedback, head-level and drop-off detection, GPS and wireless features are outside the current scope and are discussed as future work.

1.6 Organisation of the Thesis

Chapter 2 reviews existing aids and the sensing principles. Chapter 3 sets out the requirements and the architecture. Chapters 4 to 6 explain the hardware, the circuit and the microcontroller. Chapter 7 explains the software. Chapter 8 derives the physics and mathematics. Chapters 9 and 10 describe how the project was built and tested. Chapters 11 to 13 discuss results, limitations, future work and conclusions. The appendices give the full code, the costs, a glossary and quick viva answers.

2. Background and Related Work

2.1 Existing Mobility Aids

Aid	Strength	Limitation
White cane	Cheap, simple, reliable	Contact-only; no distance or water information
Guide dog	Excellent navigation and judgement	Expensive, long training, limited availability, needs care
Commercial smart canes	Electronic early warning, sometimes haptic feedback	High cost, often imported, hard to repair locally

2.2 Distance-Sensing Options

Infrared (IR) sensors are cheap but affected by sunlight and by the colour and reflectivity of the object, and they have a short range. **LiDAR** is accurate but costly. **Ultrasonic** sensors measure the time of flight of sound, work in darkness, are unaffected by colour and cost very little. This makes them the usual choice for low-cost travel aids, with the trade-off that soft or angled surfaces reflect sound poorly.

2.3 Water-Sensing Options

A **resistive** probe measures conductivity between two exposed tracks; it is cheap and gives an analog value but corrodes over time. A **capacitive** probe senses the change in dielectric constant without exposed metal; it lasts longer but costs more. A **float switch** needs standing water and is unsuitable at a cane tip. The resistive probe was chosen for cost and simplicity, with the capacitive probe noted as future work.

2.4 From Smart Shoe to Smart Cane

The first concept was a smart shoe with the same sensors. It was moved to a cane for four reasons: (1) the cane tip reaches water before the foot does; (2) a sensor on the shaft is higher and steadier than one on a shoe that rises and tilts with every step; (3) the cane has space for the battery and electronics away from the ground and sweat; and (4) blind users already know how to use a cane.

3. System Requirements and Design

3.1 Requirements

Type	Requirement
Functional	Detect obstacles under 50 cm; show distance through the beep rate; detect water with priority; indicate power-on status and low battery.
Non-functional	Low cost (under ₹900); light and portable; one switch; response much faster than walking speed; repairable with common parts.

3.2 Architecture

The system follows the classic embedded **input → processing → output** structure. The two sensors are inputs, the Arduino Nano is the processor, the buzzer is the output, and the battery and switch form the power subsystem.

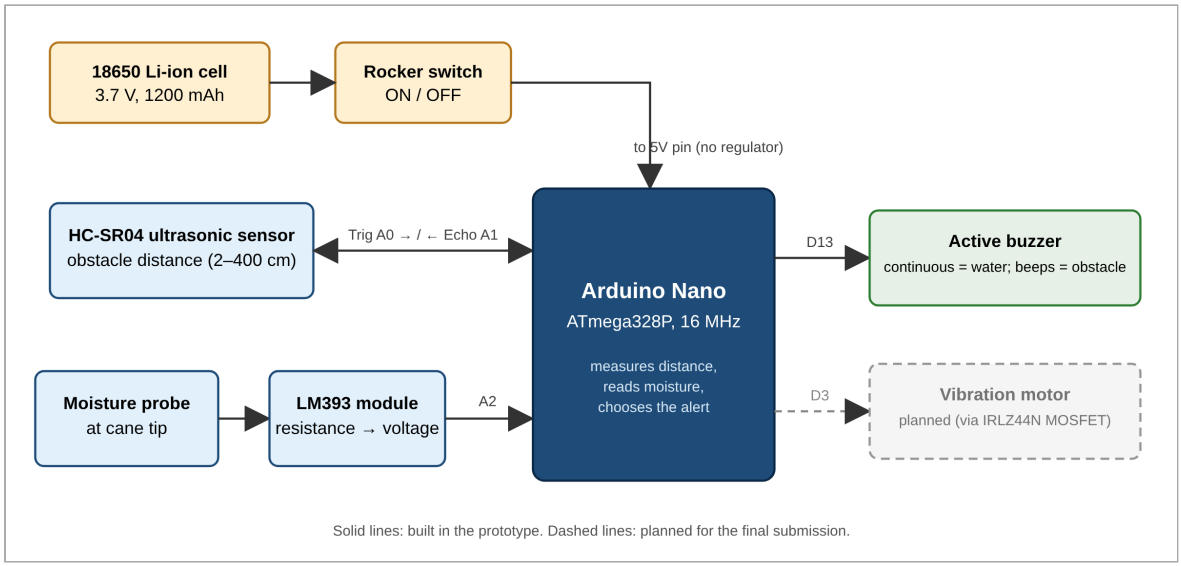


Figure 3.1: Block diagram (actual pins)

3.3 Key Design Decisions

Decision	Alternatives	Reason
Arduino Nano	Uno, ESP32, Raspberry Pi	Small (18 × 45 mm), cheap, 5 V logic, built-in ADC. A Pi is overkill in cost, power and boot time; Wi-Fi isn't needed.
HC-SR04 ultrasonic	IR, LiDAR	Cheap, 2–400 cm, works in the dark, unaffected by colour.
Resistive probe + LM393	Capacitive probe	Lowest cost; gives an analog value.
Reading A0, not D0	Comparator D0	The threshold is set in software and visible for calibration.
Active buzzer	Passive buzzer, speaker	Sounds with a simple HIGH; no tone generation needed.

Water before obstacle	Obstacle first	The tip is already at the hazard; falling or slipping is the greater risk.
Battery to 5V pin	VIN, two cells	One cell (3.7 V) is too low for the VIN regulator (about 7 V needed).
Non-blocking firmware	`delay()`-based loop	Water is detected within milliseconds, even during a beep.
Header pins + jumpers, soldered power	Breadboard	A cane vibrates constantly; breadboard contacts would loosen.

3.4 State Machine

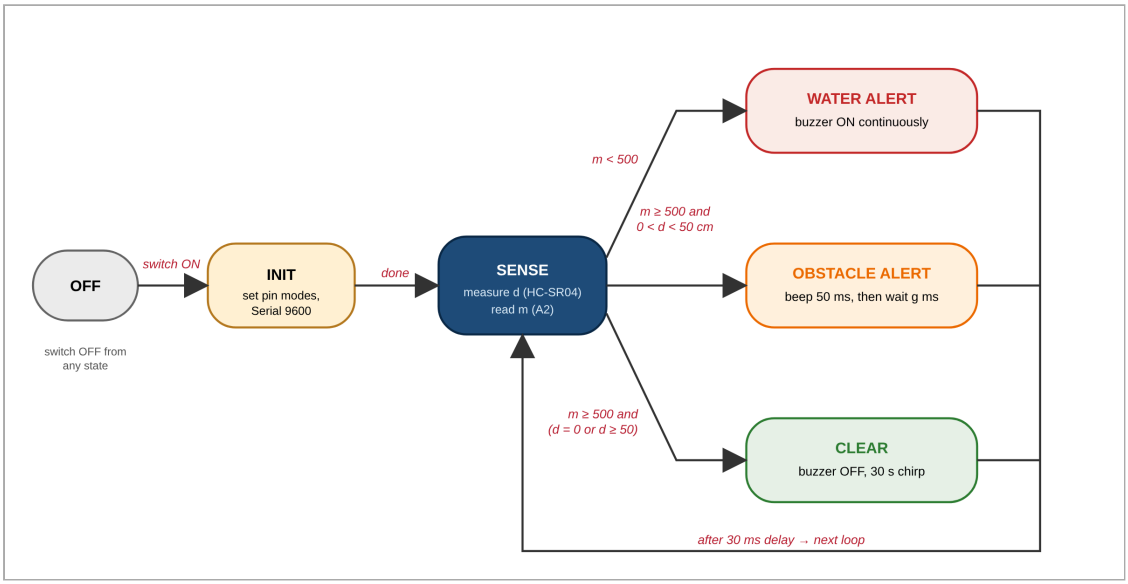


Figure 3.2: State diagram

After power-on, the system runs a self-test (INIT) and then evaluates its sensors continuously (SENSE). In each pass it is in exactly one of three states: **WATER** (moisture below threshold), **OBSTACLE** (dry and an obstacle within 2–50 cm) or **CLEAR** (otherwise). The condition for WATER is checked first, which is what gives it priority.

4. Hardware Components

4.1 Arduino Nano (ATmega328P)

The Nano is a small development board carrying an ATmega328P microcontroller at 16 MHz, a CH340 USB-to-serial chip, a 5 V/3.3 V power section and 30 pins: 14 digital I/O (6 with PWM) and 8 analog inputs. It runs the firmware, reads both sensors and drives the buzzer. Its architecture is covered in detail in Chapter 6.

4.2 HC-SR04 Ultrasonic Sensor

The module has a transmitter (T) and a receiver (R) transducer and a control circuit. When its **Trig** input is held HIGH for 10 µs, it sends a burst of eight 40 kHz pulses and sets **Echo** HIGH until the reflection returns. The Echo pulse width equals the round-trip time of the sound. Specifications: 2–400 cm range, beam angle about 15°, accuracy about 3 mm in ideal conditions. The version used (marked Trig/RX/SCL, Echo/TX/SDA) is a newer variant that also supports serial and I²C modes and a wider supply range; it is used here in the standard pulse mode.



Figure 4.1: HC-SR04 pins: Vcc, Trig, Echo, Gnd



Figure 4.2: Moisture probe at the cane tip

4.3 Moisture Probe

A fork-shaped PCB with two exposed copper tracks. In air, the resistance between the tracks is very high. Water containing dissolved salts conducts, so the resistance falls sharply when water bridges them. The probe is hot-glued at the cane tip so it touches the ground surface first.

4.4 LM393 Module

The module contains a fixed resistor that forms a voltage divider with the probe, and an LM393 comparator. Its **A0** pin gives the divider voltage (analog wetness); its **D0** pin gives a HIGH/LOW comparator output with the threshold set by the blue potentiometer. Only A0 is used, so that the threshold can be set and seen in software. Two LEDs indicate power and the D0 state.

4.5 Active Buzzer

An active buzzer contains a piezoelectric disc and its own oscillator, so applying DC (a GPIO pin set HIGH) produces a fixed tone of a few kHz. It is soldered directly onto the Nano on pin D13, which also drives the on-board "L" LED, so the LED blinks with every beep.

4.6 18650 Li-ion Cell and Holder

A cylindrical lithium-ion cell (18 mm × 65 mm), marked IMR-18650, 1200 mAh, 3.7 V, 4.44 Wh. "IMR" indicates a lithium-manganese chemistry, which is more thermally stable than older cobalt-based cells. Its voltage is 4.2 V when full, 3.7 V nominal and about 3.0 V when empty. It sits in a single-cell holder fixed with a foam strip and a cable tie.

4.7 Rocker Switch

A single-pole on/off switch (rated 6 A at 250 V AC) in series with the battery's positive line.

4.8 Electrolytic Capacitor (220 µF, 50 V)

A polarised bulk capacitor across 5V and GND. It supplies short bursts of current when the buzzer switches, limiting voltage dips that could reset the microcontroller. The stripe marks the negative lead.

4.9 Mechanical Parts

A white PVC pipe is the cane body: light, strong, cheap and an electrical insulator. Hot glue mounts the probe, the LM393 module, the Nano and the switch, and holds the wires down in dots. A white foam spacer holds the HC-SR04 flat and pointing outwards. Male header pins on the Nano and female jumper wires connect the modules.

5. Circuit, Wiring and Power

5.1 Circuit Diagram

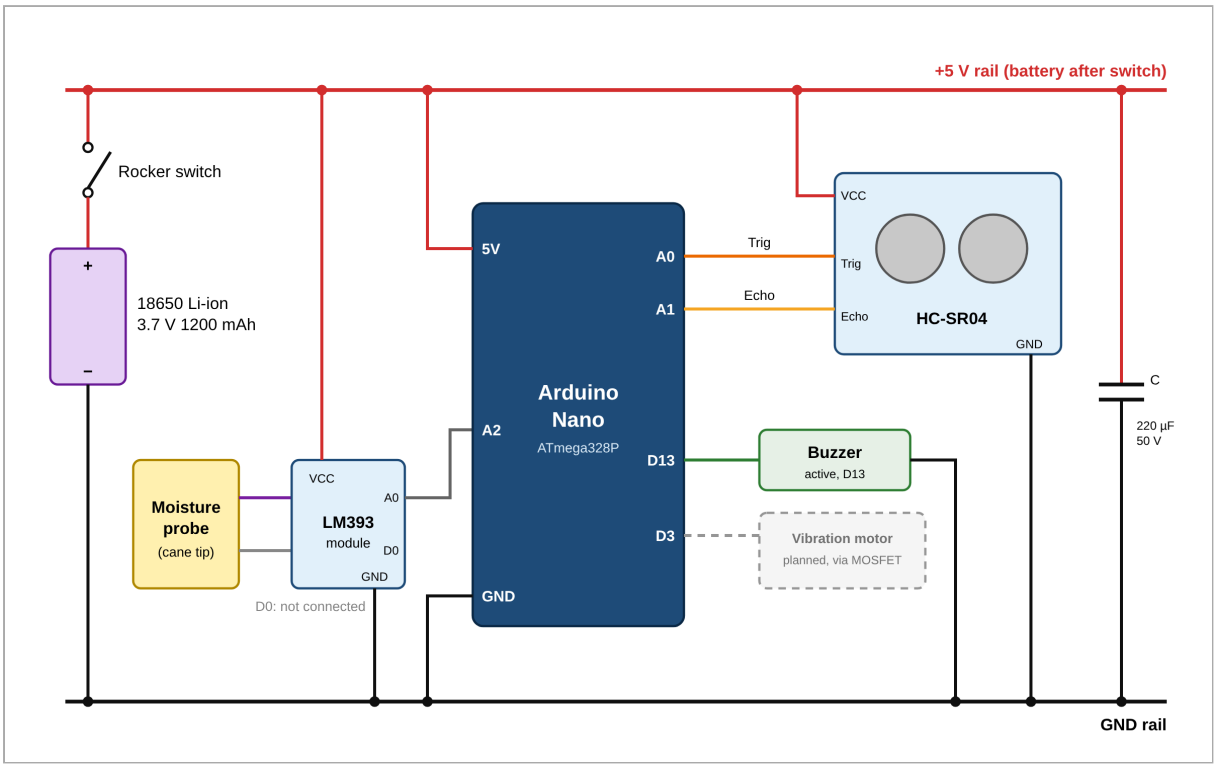


Figure 5.1: Circuit diagram (actual wiring)

5.2 Pin Connections

From	Wire	To (Nano)	Type	Reason
HC-SR04 Trig	Orange	A0	Digital out	Analog pins also work as digital pins (A0 = D14)
HC-SR04 Echo	Yellow	A1	Digital in	Echo pulse width
HC-SR04 VCC / GND	White / Black	ICSP 5V / GND	Power	The 6-pin ICSP header also carries 5V and GND
LM393 A0	Grey	A2	Analog in	Moisture voltage, 0–1023
LM393 VCC / GND	Red / Brown	5V / GND lines	Power	Module supply
LM393 D0	–	Not connected	–	Threshold is set in software
Probe	Purple / Grey	LM393 2-pin header	–	Non-polarised
Buzzer + / −	Legs	D13 / USB shell (GND)	Digital out	Soldered directly; the USB shell is ground
Battery +	Red	Switch → 5V pin	Power	Bypasses the regulator

Battery −	Black	GND	Power	Common ground
Capacitor	Legs	5V and GND	Power	Supply filtering
D3	–	Free	PWM	Reserved for a vibration motor

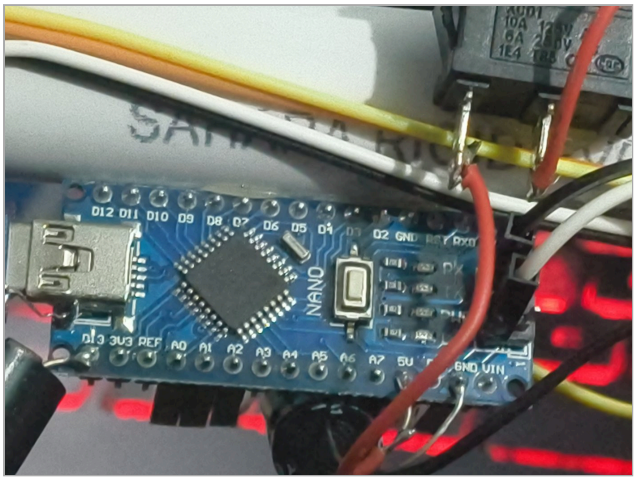


Figure 5.2: Nano pin labels, buzzer on D13, capacitor on 5V/GND

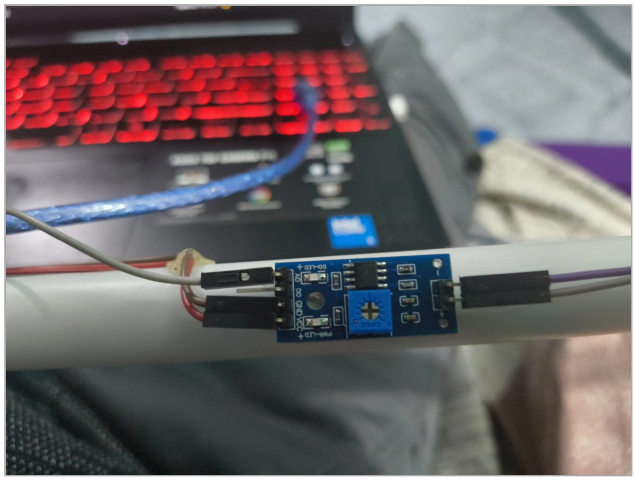


Figure 5.3: LM393 module: A0, GND, VCC used; D0 empty

5.3 Power Circuit

The cell's positive terminal goes through the rocker switch to the Nano's **5V pin**; the negative terminal goes to GND. The 5V pin connects straight to the board's 5 V rail, bypassing the on-board regulator. This is necessary because the VIN input feeds a linear regulator (AMS1117 type) that needs about 1 V of headroom, so roughly 6.5–7 V. All modules take power from the same rail, so the whole system runs at the battery voltage (3.0–4.2 V).

Consequences: (1) the supply is unregulated and falls as the cell drains; (2) the ATmega328P's guaranteed 16 MHz operation needs about 3.8 V (see 6.7), so a drained cell is outside the guaranteed range; (3) when USB is connected, USB 5 V reaches the same rail through a diode and would push current into the cell, so the switch must be OFF while USB is connected; (4) there is no over-discharge protection, which the low-battery warning in firmware partly addresses.

5.4 Grounding

All modules share one ground. Voltages are always measured relative to ground; without a common reference, the Nano could not correctly read the sensors' signal levels.

5.5 Connection Methods

Signal connections use female jumper wires on male header pins, so each module can be unplugged and tested. Power connections, the buzzer and the capacitor are soldered, because they must never come loose. No breadboard is used in the final build: a cane is swung and tapped constantly, and breadboard spring contacts would loosen.

6. The ATmega328P Microcontroller

6.1 Core Architecture

The ATmega328P is an 8-bit AVR microcontroller with a **RISC** (Reduced Instruction Set Computer) design: a small set of simple instructions, most executing in one clock cycle, giving close to 16 million instructions per second at 16 MHz. It has **32 general-purpose 8-bit registers** connected directly to the ALU, so most operations work on registers without memory access.

6.2 Harvard Architecture and Pipeline

The chip uses a **Harvard architecture**: program memory (flash) and data memory (SRAM) have separate buses. This allows a two-stage pipeline in which the next instruction is fetched while the current one executes. In contrast, a von Neumann machine uses one memory and bus for both, so instruction fetches and data accesses compete. Separate buses give simpler, more predictable timing, which suits real-time sensing.

6.3 Memory

Memory	Size	Holds in this project
Flash	32 KB (about 30 KB usable; the rest is the bootloader)	The compiled program. An earlier build of the firmware used 3,766 bytes (12%).
SRAM	2 KB	Variables (distance history, states, timers) and the stack. The earlier build used 281 bytes (13%).
EEPROM	1 KB	Unused; could store the calibrated threshold across power cycles.

6.4 General-Purpose I/O

Each I/O port has three registers: **DDRx** (direction: input or output), **PORTx** (output level, or pull-up for inputs) and **PINx** (input state). `pinMode()`, `digitalWrite()` and `digitalRead()` are Arduino functions that set and read these registers. Analog pins A0–A5 also work as digital I/O (D14–D19), which is why Trig and Echo can use A0 and A1. A pin can supply about 20 mA continuously (40 mA absolute maximum).

6.5 Analog-to-Digital Converter

The ADC is a **10-bit successive-approximation** converter with an input multiplexer. It compares the input against a binary search of internally generated voltages, one bit at a time from the most significant bit, giving a value from 0 to 1023. With a 5 V reference, one step is 5 / 1024 ≈ 4.9 mV. The Arduino core clocks the ADC at 16 MHz / 128 = 125 kHz; one conversion takes 13 ADC clock cycles, about 104 µs. The reference is **AVcc**, the supply itself. The multiplexer can also select the internal **1.1 V bandgap** reference as the input, which the firmware uses to measure the battery voltage (Section 7.6).

6.6 Timers and millis()

The chip has three hardware timers. The Arduino core configures **Timer0** to generate an overflow interrupt about every 1.024 ms; its handler increments a counter that `millis()` returns. This lets the firmware measure elapsed time without stopping the program. Timer1, a 16-bit timer, has an **input-capture** unit (pin D8) that could timestamp the echo edges in hardware, which is a possible improvement over `pulseIn()`.

6.7 Clock, Voltage and Brown-out

According to the datasheet's speed grades, the chip is guaranteed at 10 MHz from 2.7 V and at 20 MHz from 4.5 V, roughly linearly in between, so 16 MHz needs about 3.8 V. A full cell (4.2 V) is within this range; a partly drained cell is not guaranteed, although it usually still works. The **brown-out detector** resets the chip if the supply drops below its threshold (commonly about 2.7 V on Arduino boards), preventing erratic behaviour.

6.8 Serial Communication and the Bootloader

The UART (pins D0/RX and D1/TX) connects to the CH340 chip, which converts it to USB. A **bootloader** in a reserved part of flash runs briefly after reset and accepts a new program over serial. Our Nano has the newer bootloader, which uploads at 115200 baud. The application then uses the same UART at 9600 baud for the Serial Monitor. Pins D0 and D1 are therefore left free.

6.9 From Source Code to Silicon

The Arduino IDE compiles the C++ sketch with **avr-gcc** into machine code and produces an Intel **.hex** file. **avrdude** resets the board, talks to the bootloader through the CH340 and writes the program into flash. Only machine code is stored on the chip, so the original source cannot be read back from the board; the raw flash can be copied as a backup, which was done before uploading the new firmware (Section 9.5).

7. Software Design

7.1 Overall Structure

The firmware is a **bare-metal super-loop**: `setup()` runs once at power-on, then `loop()` repeats forever. There is no operating system. Unlike a simple program built around `delay()`, every task in the loop is **non-blocking**: each one checks how much time has passed using `millis()` and acts only when it is due. This makes it a **soft real-time** system, where all sensors are checked continuously and the water alarm reacts within milliseconds.

7.2 Data Flow

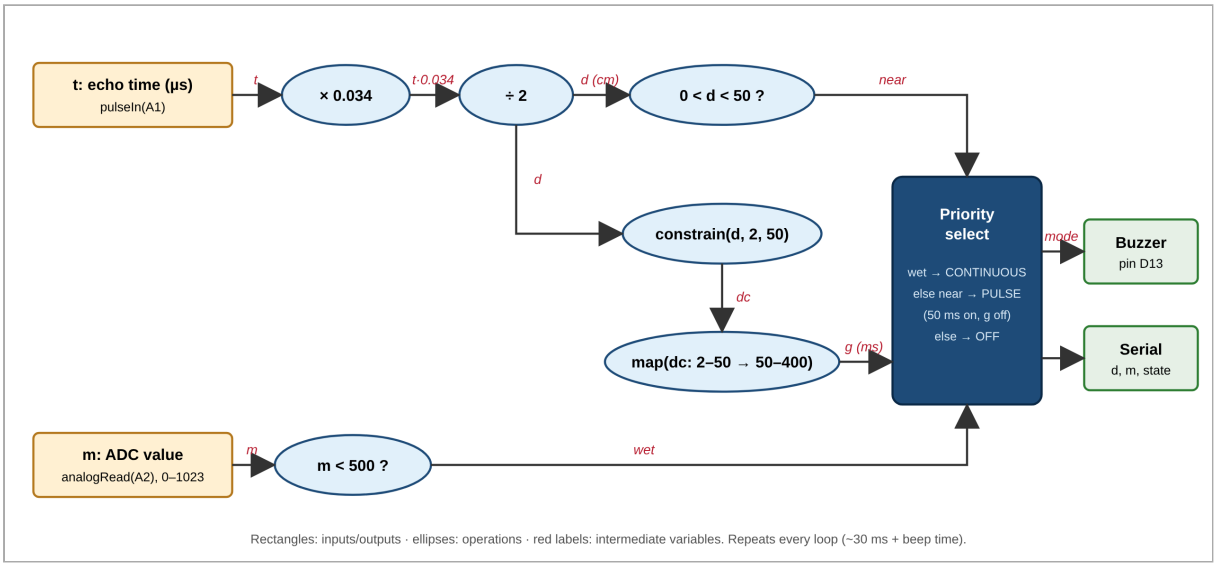


Figure 7.1: Data flow graph

The echo time *t* is converted to distance *d*, which feeds both the near-check and the beep-gap calculation. The moisture value *m* is compared with the thresholds to decide whether it is wet. A priority block combines the results to drive the buzzer. The two sensor paths do not depend on each other.

7.3 Main Loop Algorithm

1. Every 60 ms: send a ping, store the result in a 3-entry ring buffer, and set the distance to the median of the buffer.
2. Every pass: read moisture (average of 4 samples) and update the wet flag with hysteresis (wet below 500, dry above 550).
3. Every 10 s: measure the supply voltage and update the low-battery flag (below 3400 mV on, above 3600 mV off).
4. Decide the state: WATER if wet; otherwise OBSTACLE if $2 \le d < 50$ cm; otherwise CLEAR.
5. Drive the buzzer: WATER → ON continuously; OBSTACLE → 50 ms on, then off for a gap of map(d, 2–50 → 50–400 ms); CLEAR → off, except queued chirps (3 every 20 s for low battery, otherwise 1 every 30 s as a heartbeat).

6. Every 200 ms: print distance, moisture, battery voltage and state on the Serial Monitor.

7.4 Function Reference

Function	Purpose and working
`setup()`	Starts Serial at 9600 baud, sets pin directions, turns outputs off, runs `selfTest()` and initialises the timers.
`readDistanceCm()`	Holds Trig LOW for 2 µs, HIGH for 10 µs, then LOW. Measures the Echo HIGH time with `pulseIn(ECHO_PIN, HIGH, 30000)`. Returns 999 if there is no echo, otherwise $t \times 0.034 / 2$ cm.
`medianOf3()`	Returns the middle of three values, so one wrong reading (a stray echo or a miss) cannot change the output.
`readMoisture()`	Averages four `analogRead(A2)` samples to reduce noise.
`readSupplyMv()`	Sets ADMUX to measure the 1.1 V bandgap with AVcc as the reference, starts a conversion, waits, reads ADCL then ADCH, and returns 1125300 / reading (in millivolts).
`setBuzzer()`	Sets the buzzer (and D3) on or off and records the time of the change, so timing decisions can be made without `delay()`.
`beep()`	A blocking beep, used only in the self-test.
`selfTest()`	Five test pings, prints the moisture reading and supply voltage, then plays a ready chime, or one long beep if no echo was received.
`loop()`	Runs the algorithm in Section 7.3.

7.5 Filtering and Hysteresis

Median filter: ultrasonic readings occasionally miss (no echo) or catch a stray reflection, which appeared as sudden 999 cm values in testing. A median of three consecutive readings rejects any single outlier while following real changes within about 120 ms. **Averaging:** four ADC samples are averaged to smooth electrical noise. **Hysteresis:** using two thresholds (500 to turn on, 550 to turn off) prevents the water alarm from flickering when the reading sits near a single threshold.

7.6 Measuring the Battery without Extra Hardware

Because the battery feeds the supply rail directly, the supply voltage *is* the battery voltage. The ADC normally measures an input against its reference (AVcc). The firmware reverses this: it measures the fixed internal 1.1 V bandgap using AVcc as the reference. Since $reading = 1.1 \times 1023 / Vcc$, the supply is:

$$Vcc\ (mV) = 1.1 \times 1023 \times 1000 / reading = 1125300 / reading$$

The bandgap voltage varies between chips by up to about ±10%, so the constant can be calibrated against a multimeter reading.

7.7 Non-blocking Timing

Every timed action compares `now − lastTime` with an interval, for example `if (now − lastPing ≥ 60)`. Unsigned subtraction keeps this correct even when `millis()` overflows after about 49.7 days. Because nothing waits, moisture is checked on every pass of the loop, typically every few milliseconds, instead of up to about 0.5 s in a `delay()`-based design.

7.8 Firmware History

The program originally on the board was found to print *"Soil is DRY – Water needed!"* and to report a dry reading of 1023 as "100% moisture". In other words, it was adapted from a plant-watering example and its moisture logic was inverted. The new firmware described here replaces it, with a backup of the original kept (Section 9.5).

8. Physics and Mathematics

8.1 Ultrasonic Ranging

Sound travels at $v \approx 343$ m/s in air at 20 °C, which is 0.0343 cm/µs (the code uses 0.034). The Echo pulse width t is the time to the object and back, so:

$$d = v \times t / 2$$

Example: t = 2000 µs gives $d = 0.034 \times 2000 / 2 = 34$ cm. **Resolution:** 1 µs corresponds to 0.017 cm (0.17 mm); the code stores whole centimetres. **Temperature:** $v \approx 331.3 + 0.606 \cdot T$ m/s, so the error is about 0.2% per °C. **Timeout:** the maximum range of 400 cm means 800 cm of travel, or 800 / 0.034 ≈ 23.5 ms; the code waits 30 ms. **Minimum range:** the transmitter keeps ringing briefly after the burst, hiding echoes from closer than about 2 cm.

8.2 Moisture Measurement

The probe resistance R_p and the module's fixed resistor R_f form a voltage divider:

$$V_{A0} = V_{cc} \times R_p / (R_p + R_f), \quad N = 1023 \times V_{A0} / V_{ref}$$

Dry gives $R_p \to \infty$, so $N \approx 1023$; wet gives a small R_p, so N is low. Since $V_{ref} = V_{cc}$, N depends only on the resistance ratio. The measurement is therefore **ratiometric** and does not change with battery voltage. Pure water conducts poorly; puddles and mud conduct because of dissolved salts. A constant DC current through water causes **electrolysis**, which slowly corrodes the probe.

8.3 Beep-Gap Mapping

$$gap = (d - 2) \times (400 - 50) / (50 - 2) + 50 \text{ ms}$$

For d = 26 cm: (24 × 350 / 48) + 50 = 225 ms. `map()` uses integer arithmetic, so fractions are dropped.

8.4 Power and Battery Life

Estimated current is about 40 mA, so the power is 40 mA × 3.7 V ≈ 0.15 W. Theoretical runtime is 1200 mAh / 40 mA ≈ 30 h (or 4.44 Wh / 0.15 W). Real runtime will be lower. The capacitor supplies transient current $i = C \cdot dV/dt$; for example, a 20 mA spike lasting 1 ms causes a dip of only $dV = i \cdot dt / C = 0.02 \times 0.001 / 220$ µF ≈ 0.09 V.

8.5 Response Time

With the earlier `delay()`-based design, the worst case before a water alarm was 50 + 400 + 30 + 30 ≈ 510 ms. In the non-blocking design, moisture is read every pass of the loop. The longest blocking call is a `pulseIn` of up to 30 ms once every 60 ms, so the worst case is about 30 ms. At walking speed (about 1.2 m/s), 50 cm gives about 0.4 s of warning, which is why a longer range (100–150 cm) is recommended after field testing.

9. Build Methodology

9.1 Process Overview

1. Problem study and requirements (Chapters 1 and 3).
2. Concept (smart shoe), then the redesign as a cane.
3. Component selection and purchase: Vellore Robo Electronics, invoice 3293, ₹1,735 including GST.
4. Software set-up: Arduino IDE; the CH340 driver (built into Ubuntu).
5. Module testing (standard practice: each module tested on its own before assembly).
6. Mechanical assembly on the PVC pipe.
7. Wiring and soldering.
8. Firmware upload, testing and calibration.

9.2 Mechanical Assembly

From the tip upwards: the moisture probe is hot-glued at the end with its tracks exposed; the LM393 module is glued about 30 cm up; the HC-SR04 is glued to a foam spacer facing outwards; the Nano is glued flat with its USB port reachable; then come the rocker switch and the battery holder (foam strip and cable tie). The heavier parts sit near the handle, which keeps the centre of gravity close to the hand.



Figure 9.1: Completed cane, tip at the top

9.3 Soldering

Joints soldered: the male header pins on the Nano; the buzzer legs (+ to D13, − to the USB shell); the capacitor legs (5V, GND); the battery wires to the holder tabs and the switch; the switch to the Nano 5V pin; the battery negative to GND; and the joints where module power wires meet the main lines. Technique: heat the pad and the lead together for 2–3 s with a 25 W iron, then feed tin-lead solder (melting point about 183–190 °C) until it flows into a shiny cone. Flux removes oxide so the solder wets properly. A dull, grainy "cold joint" is reheated.

9.4 Uploading Firmware (Ubuntu)

1. Switch the cane OFF and connect USB; the board appears as `/dev/ttyUSB0`. If it doesn't appear, remove `brltty`; add the user to the `dialout` group.
2. In the Arduino IDE: Board = Arduino Nano, Processor = ATmega328P (new bootloader), Port = /dev/ttyUSB0.
3. Save the sketch as an `.ino` file, Verify, then Upload; confirm "Done uploading".
4. Open the Serial Monitor at 9600 baud to observe the output.

9.5 Firmware Backup

Before replacing the original program, its flash contents were saved with:

```
avrdude -p m328p -c arduino -P /dev/ttyUSB0 -b 115200 -U flash:r:cane_backup.hex:i
```

The device signature 0x1E950F confirmed an ATmega328P, and the fact that 115200 baud worked identified the newer bootloader. Extracting the text strings from the backup revealed the plant-watering messages described in Section 7.8.

10. Testing and Calibration

10.1 Test Plan

Test	Method	Expected result
Power on battery	Switch ON, no USB	Ready chime; Nano LED on
Self-test	Serial Monitor at start-up	"Ultrasonic: OK", moisture and voltage printed
Clear path	Point at open space	CLEAR; chirp every 30 s
Obstacle	Move a hand from 60 cm to 5 cm	Beeps start under 50 cm and get faster as it approaches
Water	Dip the tip about 2 cm into water	Moisture drops; continuous tone
Priority	Water and obstacle together	Continuous tone
Low battery	Run until below 3.4 V (or temporarily raise the limit)	3 chirps every 20 s

10.2 Observations So Far

With the original firmware, the Serial Monitor showed distances changing with the environment (about 81–274 cm, with occasional "999 cm" no-echo readings) and a dry moisture value of 1023. The obstacle alert was confirmed on battery power. The new firmware compiled successfully; its full test, including the water alarm and priority, is the next step.

10.3 Calibrating the Water Threshold

1. Record several dry readings and several wet readings (puddle water, wet mud) on the Serial Monitor.
2. Set `WET_ON_THRESHOLD` midway between the lowest dry and the highest wet value, and `WET_OFF_THRESHOLD` about 50 above it.
3. Re-upload and confirm the WATER state triggers reliably and clears when dried.

> **Results to be recorded:** dry reading ≈ ____ wet reading ≈ ____ threshold = ____ battery voltage on cell ≈ ____ mV

11. Results, Discussion and Limitations

11.1 Results

The prototype integrates obstacle and water sensing on a single low-cost microcontroller with prioritised audio feedback. Obstacle detection with distance-graded beeping has been demonstrated on battery power. The firmware has been redesigned to be non-blocking and filtered, with a self-test and battery monitoring, while the cane hardware costs about ₹876.

11.2 Discussion

The project shows how much useful behaviour comes from software on very simple hardware: the same sensor and buzzer give graded distance cues, a priority system, health checks and battery warnings. Building it also showed the importance of verifying the real wiring and firmware: the pins differed from the original design, and the program on the board was not the intended one.

11.3 Limitations

- The ultrasonic beam (about 15°) misses thin, soft and head-level obstacles and drop-offs.
- Water is detected only on contact; the resistive probe corrodes over time.
- The supply is unregulated (one cell on the 5V rail), and there is no hardware over-discharge protection.
- The buzzer is audible to others and may be masked by traffic noise.
- 50 cm gives limited warning at walking speed; there has been no field testing with users yet.
- Exposed electronics; no waterproof enclosure.

12. Future Work

- **Haptic feedback:** a vibration motor on D3 through the IRLZ44N logic-level MOSFET with a flyback diode; PWM for strength patterns.
- **Power:** a 3.7 V → 5 V boost converter, a TP4056 charger with DW01 protection, and the 1 A fuse.
- **Sensing coverage:** additional ultrasonic sensors angled up (head level) and down (steps, potholes).
- **Durable water sensing:** a capacitive probe, or powering the resistive probe only during readings.
- **Timing:** Timer1 input capture for the echo, freeing the CPU completely.
- **Private audio:** earphone or bone-conduction output with voice prompts (DFPlayer Mini).
- **Safety:** GPS and GSM/Bluetooth SOS messaging; EEPROM-stored calibration.
- **Product:** a PCB in a waterproof enclosure; trials with visually impaired users to tune range and alerts.

13. Conclusion

VisionGuard turns an ordinary cane into a low-cost electronic travel aid using an ATmega328P, an ultrasonic sensor and a resistive moisture probe. It warns of obstacles before contact, encodes their distance in the beep rate, and gives priority warnings for water underfoot, all from one rechargeable cell. The non-blocking firmware, with filtering, a self-test, a heartbeat and supply monitoring, makes the device more reliable and more usable without any extra hardware. The main next steps are regulated and protected power, haptic feedback, wider sensing coverage and field trials with users.

References

[1] Elecfreaks, "Ultrasonic Ranging Module HC-SR04," datasheet, 2011.

[2] Microchip Technology, "ATmega48A/PA/88A/PA/168A/PA/328/P megaAVR Data Sheet," DS40002061, 2020.

[3] Arduino, "Arduino Nano," Arduino Documentation, docs.arduino.cc/hardware/nano.

[4] Arduino, "Language Reference: pulseIn(), analogRead(), map(), millis()," arduino.cc/reference.

[5] Texas Instruments, "LMx93-N Low-Power, Low-Offset Voltage, Dual Comparators," datasheet.

[6] World Health Organization, "World Report on Vision," Geneva, 2019.

[7] AVRDUDE, "AVRDUDE – AVR Downloader/UploaDEr," user manual.

Appendix A: Complete Source Code

File: `visionguard_final.ino`. Board: Arduino Nano, Processor: ATmega328P.

```
// VisionGuard Smart Cane - obstacle + wet-surface warning
// Board: Arduino Nano (ATmega328P), new bootloader -> Processor "ATmega328P"
// Wiring: Trig A0, Echo A1, moisture A2, buzzer D13
// Team: S Darshan (25BCE2883), Tanay Agarwal (25BCE2475)
//
// What it does:
//   1. Water at the tip   -> continuous beep   (highest priority)
//   2. Obstacle < 50 cm   -> short beeps, faster when closer
//   3. Otherwise          -> silent (tiny "I'm on" chirp every 30 s)
//
// Extra features:
//   - Start-up self-test with a "ready" chime and a report on the Serial Monitor
//   - Low-battery warning: the chip measures its own supply voltage using the
//     internal 1.1 V reference (the battery feeds the 5V pin directly)
//   - Median filter on distance to remove single wrong echoes
//   - Non-blocking timing (millis) so water is detected even during a beep
//
// Calibration: open Serial Monitor at 9600, note "Soil Value" dry and wet,
// then set WET_ON_THRESHOLD between them.

// Pin Configurations (matches the actual wiring on the cane)
#define TRIG_PIN A0    // HC-SR04 Trig  (orange wire) - analog pins work as digital pins
too
#define ECHO_PIN A1    // HC-SR04 Echo  (yellow wire)
#define SOIL_PIN A2    // LM393 module A0 output (grey wire)
#define BUZZER_PIN 13  // Active buzzer, soldered on D13 (on-board "L" LED blinks with
it)
#define VIB_PIN 3      // Reserved for a vibration motor - connect it through
                       // the IRLZ44N MOSFET with a flyback diode, never directly.

// Obstacle settings
const int OBSTACLE_DISTANCE_THRESHOLD = 50;   // cm
const int MIN_DISTANCE = 2;                   // HC-SR04 cannot measure closer than ~2 cm
const unsigned long ECHO_TIMEOUT_US = 30000;  // ~5 m round trip; longer means "nothing
in range"
const unsigned long PING_INTERVAL_MS = 60;    // datasheet: at least 60 ms between pings
const int NO_ECHO_CM = 999;                   // used when no echo comes back

// Water settings (dry ~1023, wet = lower).
// Set WET_ON_THRESHOLD about halfway between your dry and wet readings.
// WET_OFF_THRESHOLD is a little higher so the alarm does not flicker (hysteresis).
const int WET_ON_THRESHOLD = 500;
const int WET_OFF_THRESHOLD = 550;
const int MOISTURE_SAMPLES = 4;               // readings averaged to reduce noise

// Battery settings (millivolts). One 18650 cell: 4.2 V full, ~3.0 V empty.
const long LOW_BATTERY_MV = 3400;             // start warning below this
const long BATTERY_OK_MV = 3600;              // stop warning above this
const unsigned long BATTERY_CHECK_MS = 10000; // check every 10 s
const unsigned long BATTERY_WARN_MS = 20000;  // warning chirps every 20 s

// Heartbeat: short chirp so a blind user knows the cane is still on
const bool HEARTBEAT_ENABLED = true;
const unsigned long HEARTBEAT_MS = 30000;

// Beep timing
const unsigned long BEEP_ON_MS = 50;          // obstacle beep length
const unsigned long CHIRP_ON_MS = 30;         // heartbeat / battery chirp length
const unsigned long CHIRP_OFF_MS = 120;       // gap between chirps
```

```
enum State { CLEAR, OBSTACLE, WATER };

int distanceHistory[3] = {NO_ECHO_CM, NO_ECHO_CM, NO_ECHO_CM};
int historyIndex = 0;
int distance = NO_ECHO_CM;
int soilMoisture = 1023;
bool isWet = false;
bool lowBattery = false;
long batteryMv = 0;

bool buzzerOn = false;
unsigned long lastBuzzerChange = 0;
int chirpsLeft = 0;

unsigned long lastPing = 0;
unsigned long lastBatteryCheck = 0;
unsigned long lastBatteryWarn = 0;
unsigned long lastHeartbeat = 0;
unsigned long lastPrint = 0;

// ---------------- Sensors ----------------

// Returns distance in cm, or NO_ECHO_CM if no echo came back in time
int readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);
  if (duration == 0) return NO_ECHO_CM;
  // Sound travels 0.034 cm/us; divide by 2 because the pulse goes out and back
  return duration * 0.034 / 2;
}

// Middle value of the last 3 readings: one wrong echo cannot change the result
int medianOf3(int a, int b, int c) {
  if ((a <= b && b <= c) || (c <= b && b <= a)) return b;
  if ((b <= a && a <= c) || (c <= a && a <= b)) return a;
  return c;
}

// Average a few ADC readings so one noisy value cannot trigger the alarm
int readMoisture() {
  long total = 0;
  for (int i = 0; i < MOISTURE_SAMPLES; i++) {
    total += analogRead(SOIL_PIN);
  }
  return total / MOISTURE_SAMPLES;
}

// Measures the chip's own supply voltage (= battery voltage, since the cell
// feeds the 5V pin). Trick: use the supply as the ADC reference and measure
// the internal 1.1 V bandgap. Vcc = 1.1 V x 1023 / reading.
long readSupplyMv() {
  ADMUX = _BV(REFS0) | _BV(MUX3) | _BV(MUX2) | _BV(MUX1);  // ref = AVcc, input = 1.1 V
bandgap
  delay(2);                                                // let the reference settle
  ADCSRA |= _BV(ADSC);                                     // start conversion
  while (bit_is_set(ADCSRA, ADSC)) {}                      // wait until done
  int low = ADCL;                                          // read ADCL first, then ADCH
  int high = ADCH;
  long reading = (high << 8) | low;
  if (reading == 0) return 0;
  return 1125300L / reading;                               // 1.1 x 1023 x 1000
```

```
}

// ---------------- Buzzer ----------------

void setBuzzer(bool on, unsigned long now) {
  buzzerOn = on;
  lastBuzzerChange = now;
  digitalWrite(BUZZER_PIN, on ? HIGH : LOW);
  digitalWrite(VIB_PIN, on ? HIGH : LOW);
}

// Blocking beep, used only during start-up
void beep(int onMs, int offMs) {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(onMs);
  digitalWrite(BUZZER_PIN, LOW);
  delay(offMs);
}

// ---------------- Start-up self-test ----------------

void selfTest() {
  Serial.println("=== VisionGuard self-test ===");

  int echoes = 0;
  for (int i = 0; i < 5; i++) {
    if (readDistanceCm() != NO_ECHO_CM) echoes++;
    delay(PING_INTERVAL_MS);
  }
  Serial.print("Ultrasonic: ");
  Serial.println(echoes > 0 ? "OK" : "no echo - check wiring or point at a wall");

  Serial.print("Moisture reading: ");
  Serial.println(readMoisture());

  batteryMv = readSupplyMv();
  Serial.print("Supply voltage: ");
  Serial.print(batteryMv);
  Serial.println(" mV");

  if (echoes == 0) {
    beep(500, 200);  // one long beep = sensor problem
  } else {
    beep(60, 80);    // "ready" chime: three rising-speed beeps
    beep(60, 60);
    beep(120, 0);
  }
  Serial.println("=== Ready ===");
}

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(VIB_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(VIB_PIN, LOW);

  selfTest();

  unsigned long now = millis();
  lastHeartbeat = now;
  lastBatteryCheck = now;
  lastBatteryWarn = now;
```

```
}

// ---------------- Main loop (non-blocking) ----------------

void loop() {
  unsigned long now = millis();

  // 1. Distance: one ping every 60 ms, median of the last 3
  if (now - lastPing >= PING_INTERVAL_MS) {
    lastPing = now;
    distanceHistory[historyIndex] = readDistanceCm();
    historyIndex = (historyIndex + 1) % 3;
    distance = medianOf3(distanceHistory[0], distanceHistory[1], distanceHistory[2]);
  }

  // 2. Water, with hysteresis - read every loop so it reacts immediately
  soilMoisture = readMoisture();
  if (soilMoisture < WET_ON_THRESHOLD) {
    isWet = true;
  } else if (soilMoisture > WET_OFF_THRESHOLD) {
    isWet = false;
  }

  // 3. Battery, every 10 s, with hysteresis
  if (now - lastBatteryCheck >= BATTERY_CHECK_MS) {
    lastBatteryCheck = now;
    batteryMv = readSupplyMv();
    if (batteryMv < LOW_BATTERY_MV) lowBattery = true;
    else if (batteryMv > BATTERY_OK_MV) lowBattery = false;
  }

  // 4. Decide the state - water first, so it always has priority
  State state = CLEAR;
  if (isWet) {
    state = WATER;
  } else if (distance >= MIN_DISTANCE && distance < OBSTACLE_DISTANCE_THRESHOLD) {
    state = OBSTACLE;
  }

  // 5. Drive the buzzer without delay()
  if (state == WATER) {
    chirpsLeft = 0;
    if (!buzzerOn) setBuzzer(true, now);              // continuous tone
  }
  else if (state == OBSTACLE) {
    chirpsLeft = 0;
    int clamped = constrain(distance, MIN_DISTANCE, OBSTACLE_DISTANCE_THRESHOLD);
    unsigned long gap = map(clamped, MIN_DISTANCE, OBSTACLE_DISTANCE_THRESHOLD, 50, 400);
    if (buzzerOn && now - lastBuzzerChange >= BEEP_ON_MS) {
      setBuzzer(false, now);                           // end of beep
    } else if (!buzzerOn && now - lastBuzzerChange >= gap) {
      setBuzzer(true, now);                            // start next beep
    }
  }
  else {
    // Path clear: queue chirps for low battery (3) or heartbeat (1)
    if (lowBattery && chirpsLeft == 0 && now - lastBatteryWarn >= BATTERY_WARN_MS) {
      lastBatteryWarn = now;
      chirpsLeft = 3;
    } else if (HEARTBEAT_ENABLED && chirpsLeft == 0 && now - lastHeartbeat >=
HEARTBEAT_MS) {
      lastHeartbeat = now;
      chirpsLeft = 1;
    }

    if (buzzerOn && now - lastBuzzerChange >= CHIRP_ON_MS) {
```

```
      setBuzzer(false, now);
      if (chirpsLeft > 0) chirpsLeft--;
    } else if (!buzzerOn && chirpsLeft > 0 && now - lastBuzzerChange >= CHIRP_OFF_MS) {
      setBuzzer(true, now);
    }
  }

  // 6. Status on the Serial Monitor, 5 times a second
  if (now - lastPrint >= 200) {
    lastPrint = now;
    Serial.print("Distance: ");
    if (distance == NO_ECHO_CM) Serial.print("--");
    else Serial.print(distance);
    Serial.print(" cm | Soil Value: ");
    Serial.print(soilMoisture);
    Serial.print(" | Battery: ");
    Serial.print(batteryMv);
    Serial.print(" mV | ");
    if (state == WATER) Serial.print("ALERT: WATER");
    else if (state == OBSTACLE) Serial.print("ALERT: OBSTACLE");
    else Serial.print("CLEAR");
    if (lowBattery) Serial.print(" | LOW BATTERY");
    Serial.println();
  }
}
```

Appendix B: Bill of Materials

Item	Qty	Cost (₹, incl. GST)
Arduino Nano (ATmega328P)	1	250
HC-SR04 ultrasonic sensor	1	105
Soil-moisture probe + LM393 module	1	45
18650 Li-ion cell (one fitted, one spare)	2	130
Cell holder	1	30
Rocker switch	1	15
Electrolytic capacitors	4	45
Jumper wires (3 sets), berg strips (5), wire (3)	–	146
Nano USB cable	1	40
Active buzzer (estimated)	1	20
PVC pipe, hot glue, cable ties (estimated)	–	50
Cane hardware total		**876**
Tools (iron, solder, flux, breadboard)	–	660
Parts for future work (MOSFET, fuse + holder, JST, resistors, 0.1 µF)	–	269
Total spending		**1,805**

Appendix C: A–Z Glossary

Term	Meaning
ADC	Analog-to-digital converter: voltage → number (0–1023).
Active buzzer	Buzzer with a built-in oscillator; DC makes sound.
AVcc	The ADC's supply pin; our default reference.
Bandgap (1.1 V)	Fixed internal reference, used to measure the battery.
Baud	Serial speed in bits/s: 9600 for the monitor, 115200 for upload.
Bootloader	Small program in flash that accepts code over USB serial.
Brown-out detector	Resets the chip if the voltage drops too low.
Capacitor	Stores charge; smooths supply dips ($i = C \cdot dV/dt$).
CH340	USB-to-UART chip on clone Nanos.
Cold joint	Dull, weak solder joint from too little heat.
Comparator	Compares two voltages → HIGH/LOW (the LM393).
constrain()	Clamps a value into a range.
delay()	Blocking wait; avoided in our loop.
Echo pin	HC-SR04 output; HIGH for the round-trip time.
EEPROM	1 KB memory that survives power-off.
Electrolysis	DC through water corrodes the probe.
Flash	32 KB program memory.
Flux	Removes oxide so solder flows.
Flyback diode	Absorbs a motor's switch-off voltage spike.
GND	0 V reference, common to all parts.
Harvard architecture	Separate program and data memory/buses.
Header pins	Metal pins soldered on the board for jumpers.
Hysteresis	Two thresholds (on/off) to stop flicker.
ICSP header	6-pin programming header; also gives 5V/GND.
IRLZ44N	Logic-level MOSFET for the future motor.
Jumper wire	Plug-in wire (female end fits header pins).
LM393	Dual comparator on the moisture module.
loop()	Function that repeats forever.
map()	Linear scaling from one range to another.
Median filter	Uses the middle of several readings; rejects outliers.
millis()	Time since start in ms (Timer0 interrupt).
MOSFET	Transistor switch for larger currents.

Non-blocking	Code that never waits, so everything keeps running.
pulseIn()	Measures pulse width in µs; blocking.
PWM	Fast on/off switching to vary power (D3 supports it).
Ratiometric	Reading depends on a ratio, not the supply value.
RISC	Simple instructions, mostly 1 clock each.
Serial / UART	Two-wire serial link (RX/TX) to the laptop.
setup()	Function that runs once at power-on.
Soft real-time	Should respond fast; a small delay isn't catastrophic.
SRAM	2 KB working memory for variables.
Super-loop	Bare-metal design: setup once, loop forever, no OS.
Trig pin	HC-SR04 input; a 10 µs HIGH starts a measurement.
Ultrasound	Sound above 20 kHz (we use 40 kHz).
VCC / 5V pin	Supply rail; our battery feeds it directly.
VIN	Input to the on-board regulator; needs about 7 V+.
Voltage divider	Two resistances; the middle voltage depends on their ratio.

Appendix D: Quick Viva Answers

Question	Crisp answer
What is the project?	A smart cane: faster beeps as obstacles come within 50 cm; a continuous beep when the tip touches water, which has priority.
Why divide by 2?	Sound travels there and back.
Why water first?	The tip is already at the hazard; slipping or falling is the bigger risk.
Why A0/A1 for the ultrasonic?	Analog pins also work as digital pins (D14/D15).
Why the 5V pin, not VIN?	VIN's regulator needs about 7 V; one cell is 3.7 V.
Why millis(), not delay()?	Non-blocking: water is checked every pass, even mid-beep.
How is the battery measured?	The 1.1 V bandgap against AVcc: Vcc = 1.1 × 1023 ÷ reading.
Why 30 ms timeout / 60 ms ping?	400 cm round trip ≈ 23.5 ms; the datasheet asks for at least 60 ms between pings.
Why a median filter?	It rejects a single wrong echo (e.g. a 999 cm miss).
Why hysteresis?	Two thresholds stop the alarm flickering near the limit.
Harvard architecture?	Separate flash and SRAM buses; fetch and execute overlap.
ADC type?	10-bit successive approximation, about 4.9 mV/step, about 104 µs/conversion.
16 MHz on 3.7 V?	Guaranteed from about 3.8 V; a drained cell is marginal, so a boost converter is planned.
Why no breadboard?	Vibration; soldered power joints and jumpers on header pins are reliable.
Biggest limitation?	The narrow beam misses head-level objects; water is detected only on contact.
Next step?	A vibration motor via the MOSFET, regulated and protected power, field trials.